principle-viz: Diagrams for Principles of Economics

Pin Yue Sung[*†] Ling Tak Douglas Chung[*]

2026-10-23 · v0.12.1 · ·

Contents

1 Introduction 2
1.1 Scope 2
1.2 Reading guide 2
2 Installation 3
2.1 Requirements 3
2.2 Installing uv 3
2.3 Installing principle-viz 3
2.4 Installing the command-line tool 4
2.5 Development setup 4
2.6 Verifying the installation 4
3 Quick start 4
3.1 A minimal example 4
3.2 Step by step 5
3.3 Results 6
3.4 Input errors and solver failures 6
4 Linear markets 6
4.1 Lines 6
4.2 Equilibrium 7
4.3 Comparative statics 7
4.4 Errors 9
5 Discrete markets 9
5.1 Schedules 9
5.2 Equilibrium 9
5.3 Figure 10
6 Market curves from individuals 11
6.1 Summing linear curves 11
6.2 Summing discrete schedules 12
6.3 Aggregation figures 12
7 Elasticity and total revenue 13
7.1 Price elasticity 13
7.2 Total revenue 14
8 Welfare 14
8.1 Market outcomes 14
8.2 Surplus 15
8.3 Figures 16
8.4 Deadweight-loss reports 16
9 Taxes and subsidies 17
9.1 Scenarios 17
9.2 Solving 17
9.3 Figures 18
9.4 Subsidies 19
10 Price controls 20
11 International trade 22
12 Market failures 24
12.1 Externalities 24
12.2 Common resources 25
12.3 Public goods 26
13 Labor and loanable funds 26
13.1 Minimum wage 26
13.2 Loanable funds 28
14 Production possibilities 28
14.1 Comparative advantage 30
15 Figures 30
15.1 MarketFigure 30
15.2 Labels and layers 32
15.3 Palettes and themes 33
15.4 Settings file 34
15.5 Canvases 35
16 Command-line interface 35
16.1 Market arguments 35
16.2 Commands 36
Change history 38
Command Index 40
References 42

[*]Department of International Business, National Chengchi University, Taipei, Taiwan.
[†]Corresponding author. Email: contact@econ-viz.org

1 Introduction

principle-viz is a Python package for the diagrams of a principles-of-economics course: supply and demand, equilibrium and its shifts, taxes and subsidies, price controls, welfare, international trade, market failures, factor markets and the production possibilities frontier. Each topic has a calculation, which returns numbers, and a figure drawn in textbook conventions.

Figures are written as PNG, SVG or PDF; the command-line tool prints calculation results as JSON.

principle-viz belongs to the EconViz family of packages and draws with mosaickit, a domain-independent scene and drawing library. Its sibling utility-viz is a microeconomics package covering utility models, optimal-bundle solving, indifference curves, budget lines, consumer equilibrium, demand curves and Edgeworth boxes; principle-viz does not include these.

1.1 Scope

principle-viz handles linear markets, and its features fall into four parts: markets, policy, applications and tools. Calculations and figures can be used separately: a result can be printed, written as JSON or passed to `MarketFigure` to be drawn.

1.2 Reading guide

The manual is organized in four parts: markets, policy, applications and tools (see Table 1).

Table 1 Chapter guide

Part	Topic	Description	Section
Markets	Linear markets	Lines, equilibrium and shifts	Section 4
	Discrete markets	Unit-by-unit demand and supply schedules	Section 5
	Market curves from individuals	Horizontal sum of individual curves	Section 6
	Elasticity and total revenue	Point and arc elasticity, total revenue curve	Section 7
Policy	Welfare	Consumer surplus, producer surplus and deadweight loss	Section 8
	Taxes and subsidies	Tax wedge and subsidy cost	Section 9
	Price controls	Price ceilings, price floors and shortages	Section 10
Applications	International trade	Free trade, tariffs and import quotas	Section 11
	Market failures	Externalities, common resources and public goods	Section 12

Part	Topic	Description	Section
	Labor and loanable funds	Minimum wage and government borrowing	Section 13
	Production possibilities	Opportunity cost, growth and comparative advantage	Section 14
Tools	Figures	Labels, layers and palettes	Section 15
	Command line	Calculation results as JSON	Section 16

On first use, read Section 3, Section 4 and Section 15 in that order. Each later chapter covers one topic: the calculation first, then the figure. For the options of a specific command, go to Section 16.

The scripts that draw the figures of this manual come as one zip, sorted by topic: principle-viz-examples.zip.

2 Installation

2.1 Requirements

principle-viz requires Python 3.10 or later[1]. Its only runtime dependency is mosaickit, which draws the figures through matplotlib; the calculations use only the standard library.

2.2 Installing uv

The commands in this manual use uv[2]. Existing projects can keep using pip, pipx or Poetry; the package API does not depend on the tool.

Run the command for the operating system.

macOS and Linux

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

Windows

```
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps
1 | iex"
```

2.3 Installing principle-viz

Create a project and add the package:

```
uv init my-diagrams
cd my-diagrams
uv add principle-viz
uv run python main.py
```

[1] The Python website provides installers for every operating system: `https://www.python.org/downloads/`.

[2] uv is a fast Python package and project manager by Astral that also manages Python versions. Installation and full documentation: `https://docs.astral.sh/uv/`.

Save the example of Section 3 as `main.py` in the project directory before running the commands above. `uv add` records the project dependency and `uv run` uses the project's Python environment. Installing and running must use the same environment.

To pin the version this manual describes, name it when adding the dependency:

```
uv add "principle-viz==0.12.1"
```

In an existing virtual environment, use pip:

```
python -m pip install -U principle-viz
```

The distribution is named `principle-viz`; the Python import name is `principle_viz`:

```
import principle_viz
from principle_viz import solve_equilibrium, MarketFigure
```

An `import` statement cannot contain a hyphen. When `ModuleNotFoundError` occurs, check that the interpreter running the program belongs to the environment where the package was installed.

The package was published as `principle-econ` before version 0.10.0. That distribution receives no further updates; install `principle-viz` and change imports from `principle_econ` to `principle_viz`.

2.4 Installing the command-line tool

To use only the command-line interface, install principle-viz as a standalone tool (Section 16):

```
uv tool install principle-viz
principle-viz equilibrium --demand-intercept 10 --demand-slope -1 \
                          --supply-intercept 2 --supply-slope 1
```

This installs the tool in its own environment. Importing the package in Python code still requires `uv add principle-viz` in that project. Calling `uv run principle-viz` inside a project makes the command line and the Python code use the same version.

2.5 Development setup

```
git clone https://github.com/EconViz/principle-viz.git
cd principle-viz
uv sync
uv run pytest -q
uv run ruff check src tests examples/scripts
```

The test suite enforces 90% statement coverage. The example scripts write every figure of the project gallery to `examples/output/`:

```
uv run python examples/scripts/run_all.py
```

2.6 Verifying the installation

This command checks that Python can import the solver and plotting interface:

```
uv run python -c "import principle_viz; print('OK')"
```

`uv run principle-viz --help` checks the command-line tool and lists all commands. In a server or other environment without a display, write files with `save()`.

3 Quick start

3.1 A minimal example

This chapter uses one market, which later chapters reuse:

$$\text{demand:} \quad p = 10 - Q,$$
$$\text{supply:} \quad p = 2 + Q.$$

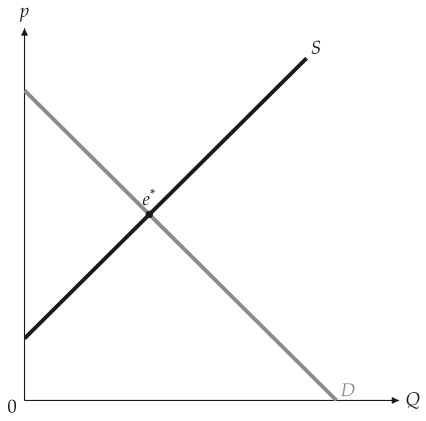


Figure 1
The market of this chapter

Example 1

```
from principle_viz import MarketFigure, line_from_inverse, solve_equilibrium

demand = line_from_inverse(10.0, -1.0)   # p = 10 - Q
supply = line_from_inverse(2.0, 1.0)     # p = 2 + Q
eq = solve_equilibrium(demand, supply)
print(eq.q_star, eq.p_star)              # 4.0 6.0

fig = MarketFigure(x_max=12, y_max=12, title="Basic Equilibrium")
fig.add_curves(demand, supply, q_max=10)
fig.add_equilibrium(eq)
fig.finalize()
fig.save("basic_equilibrium.png")
```

$10 - Q = 2 + Q$ gives $Q^* = 4$ and $p^* = 6$. The output is shown in Figure 1.

3.2 Step by step

Every figure goes through four steps: describe the market, solve, draw, and finish and save.

`line_from_inverse`

`line_from_inverse(`⟨*float*⟩`, `⟨*float*⟩`)`

Describe the market. `line_from_inverse(a, b)` builds the line $p = a + bQ$ (Section 4).

`solve_equilibrium`

`solve_equilibrium(`⟨*demand*⟩`, `⟨*supply*⟩`)`

Solve. `solve_equilibrium()` and the other solvers return a frozen dataclass of numbers. Nothing is drawn at this step.

`MarketFigure`

`MarketFigure(x_max=`⟨*float*⟩`, y_max=`⟨*float*⟩`, ...)`

Draw. `MarketFigure` is a square diagram with a price axis and a quantity axis. The axis ranges only set the visible area and do not enter any calculation; when the equilibrium point is missing from the figure, check whether `eq.q_star` and `eq.p_star` exceed the ranges, then raise the axis limits.

`MarketFigure.add_*`

The `add_*` methods take curves and results and add the matching layers. Each method returns the figure, so calls can be chained.

`MarketFigure.finalize`
`MarketFigure.save`

Finish and save. `finalize()` hides guide lines that would cut a shaded area in two; `save()` writes PNG, SVG or PDF according to the file extension and creates missing directories (Section 15).

Calculation and drawing are independent: a result can be printed, compared or exported without drawing it, and the same result can be drawn on several figures.

3.3 Results

Results are immutable dataclasses whose fields are floats, strings and tuples. `solve_equilibrium()` returns an `EquilibriumResult`:

`q_star` *float* — Equilibrium quantity Q^*.

`p_star` *float* — Equilibrium price p^*.

`is_valid_market` *bool* — `False` when the equilibrium quantity is negative; a negative price only adds a note.

`notes` *tuple[str, ...]* — Notes on special cases of the solution.

3.4 Input errors and solver failures

`PrincipleVizError` — Every exception the package raises derives from `PrincipleVizError` in `principle_viz.exceptions`; catching it handles all of them (see Section 4.4).

Parallel lines have no equilibrium. `solve_equilibrium()` raises `ParallelLinesError` and never returns an invalid result:

```
from principle_viz import line_from_inverse, solve_equilibrium
from principle_viz.exceptions import PrincipleVizError

demand = line_from_inverse(10.0, -1.0)
supply = line_from_inverse(8.0, -1.0)   # parallel to demand

try:
    eq = solve_equilibrium(demand, supply)
except PrincipleVizError as error:
    print(type(error).__name__)
else:
    print(eq.q_star, eq.p_star)

# ParallelLinesError
```

A negative equilibrium quantity raises no exception; it sets `is_valid_market` to `False`. Check `is_valid_market` and `notes` before using a result.

4 Linear markets

4.1 Lines

A demand or supply curve is a straight line in inverse form:

$$p = a + bQ$$

where a is the price intercept and b the slope ($b < 0$ for demand, $b > 0$ for supply). Price is always on the vertical axis and quantity on the horizontal one, following Marshall (1890). Discrete unit schedules (Section 5) and market curves summed from individual curves (Section 6) are not single straight lines.

`Line` — `Line.from_inverse(⟨float⟩, ⟨float⟩)`
`Line.from_standard(⟨float⟩, ⟨float⟩, ⟨float⟩)`

A straight line in the price–quantity plane, stored as $Ap + BQ + C = 0$ so that horizontal and vertical lines are allowed. `from_inverse(a, b)` builds $p = a + bQ$; `from_standard(A, B, C)` builds $Ap + BQ + C = 0$. The package root also exports the two constructors as `line_from_inverse()` and `line_from_standard()`.

`p_at(q)` Price at quantity q.

`q_at(p)` Quantity at price p.

`p_intercept()` Price at $Q = 0$ (the choke price of a demand curve).

`q_intercept()` Quantity at $p = 0$.

`slope()` $\mathrm{d}p/\mathrm{d}Q$.

`to_inverse()` `(a, b)` for $p = a + bQ$.

`shifted()` A new line with `delta_intercept` (Δa) added to the intercept and `delta_slope` (Δb) added to the slope.

```python
from principle_viz import line_from_inverse, line_from_standard

demand = line_from_inverse(10.0, -1.0)
print(demand.q_at(4), demand.p_at(3))         # 6.0 7.0
print(demand.p_intercept(), demand.q_intercept())  # 10.0 10.0
print(line_from_standard(1, 1, -10).to_inverse())  # (10.0, -1.0)
```

$(A, B, C) = (1, 1, -10)$ is equivalent to $p = 10 - Q$.

A horizontal line has no $Q(p)$ and a vertical line has no $p(Q)$; the corresponding methods raise `NonInvertibleLineError`.

4.2 Equilibrium

`solve_equilibrium` `solve_equilibrium(`⟨*demand*⟩`,` ⟨*supply*⟩`)`

The intersection of two lines, returned as an `EquilibriumResult` (Section 3). Parallel lines raise `ParallelLinesError`, identical lines `CoincidentLinesError`.

```python
from principle_viz import solve_equilibrium

eq = solve_equilibrium(demand, line_from_inverse(2.0, 1.0))
print(eq)
# EquilibriumResult(q_star=4.0, p_star=6.0,
#                   is_valid_market=True, notes=())
```

$10 - Q = 2 + Q$ gives an equilibrium quantity of 4 and an equilibrium price of 6; `is_valid_market` is `True` and `notes` is empty.

4.3 Comparative statics

`ShiftSpec` `ShiftSpec(delta_intercept=0.0, delta_slope=0.0)`

`ShiftScenario` `ShiftScenario(demand_shift=None, supply_shift=None)`

A shift of one curve in inverse form: `delta_intercept` moves it up (> 0) or down, `delta_slope` rotates it. A scenario shifts demand, supply or both. Both live in `principle_viz.core.shifts`.

An increase in demand raises the demand intercept; an increase in supply lowers the supply intercept, since sellers accept a lower price at every quantity.

`comparative_statics` `comparative_statics(`⟨*demand*⟩`,` ⟨*supply*⟩`,` ⟨*scenario*⟩`)`

Solve the market before and after the shift. The result has these fields:

`baseline_equilibrium` The `EquilibriumResult` before the shift.

`shifted_equilibrium` The `EquilibriumResult` after the shift.

`shifted_market` The baseline and shifted lines (`baseline_demand`, `shifted_demand`, …).

`delta_q` Change in quantity.

`delta_p` Change in price.

`direction_q` Direction of the quantity change: `"left"` or `"right"` (`"none"` when unchanged).

`direction_p` Direction of the price change: `"up"` or `"down"` (`"none"` when unchanged).

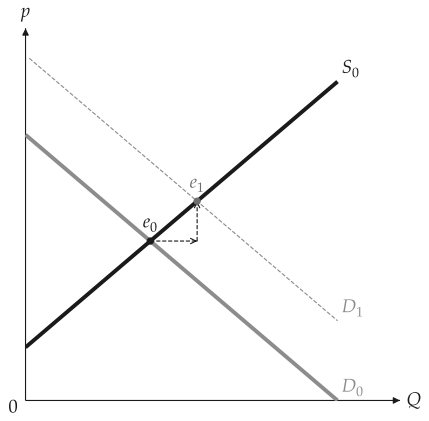


Figure 2
Demand increase

```
from principle_viz import comparative_statics
from principle_viz.core.shifts import ShiftScenario, ShiftSpec

up = ShiftScenario(demand_shift=ShiftSpec(delta_intercept=3.0))
result = comparative_statics(demand, supply, up)
new = result.shifted_equilibrium
print(new.q_star, new.p_star)                    # 5.5 7.5
print(result.direction_q, result.direction_p)  # right up
```

`MarketFigure.add_comparative_statics`

`add_comparative_statics(⟨result⟩, q_max, *, demand_label="$D_1$", supply_label="$S_1$")`

Draw the curve that moved, both equilibria and dashed arrows from the old equilibrium to the new one. Name the original curves D_0 and S_0 in `add_curves()`. An increase in demand is shown in Figure 2 and a decrease in supply in Figure 3.

```
fig = MarketFigure(x_max=12, y_max=14, title="Increase in Demand")
fig.add_curves(
    demand, supply, q_max=10,
    demand_label="$D_0$", supply_label="$S_0$",
)
fig.add_comparative_statics(result, q_max=10)
fig.finalize()
```

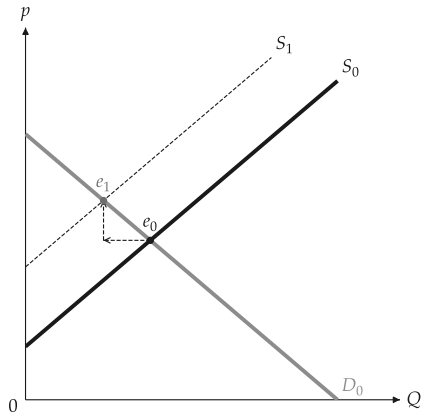


Figure 3
Supply decrease

4.4 Errors

Every exception the package raises derives from `PrincipleVizError`, in `principle_viz.exceptions`; catching it handles all of them. `PrincipleEconError`, the name before 0.10.0, is the same class. Table 2 lists when each exception is raised.

Table 2 Exceptions

Exception	Raised when
`LineError`	A line cannot be built or transformed
`NonInvertibleLineError`	A horizontal line is asked for $Q(p)$, or a vertical one for $p(Q)$
`ParallelLinesError`	Demand and supply never meet
`CoincidentLinesError`	Demand and supply are the same line
`PolicyError`	A tax, subsidy, control or trade scenario is invalid
`DiscreteMarketError`	A discrete schedule is invalid (Section 5)
`AggregationError`, `PiecewiseLinearError`	Individual curves cannot be summed, or a price is outside a piecewise curve (Section 6)
`PPFError`	A production possibilities frontier is invalid (Section 14)

5 Discrete markets

A discrete market lists units one at a time: each buyer's willingness to pay for one more unit, and each seller's cost of one more unit. Demand and supply are step functions; the equilibrium is a whole number of units, supported by a range of prices.

5.1 Schedules

`DiscreteDemand` `DiscreteDemand(⟨values⟩)`
`DiscreteSupply` `DiscreteSupply(⟨values⟩)`

A demand schedule holds marginal willingness-to-pay values, weakly decreasing from the first unit to the last; a supply schedule holds marginal costs, weakly increasing. Values out of order raise `DiscreteMarketError`.

`quantity_at(price)` Units bought or sold at `price`; a buyer or seller indifferent at the price trades.

`unit_count` Number of units in the schedule.

`combine(*schedules)` Merge several individual schedules into a market schedule (Section 6).

5.2 Equilibrium

`solve_discrete_equilibrium` `solve_discrete_equilibrium(⟨demand⟩, ⟨supply⟩, *, price_rule="midpoint")`

Trade every unit whose value is at least its cost, and find the interval of prices at which exactly that many units are bought and sold. The result has these fields:

`q_star` *int* — Units traded.

`price_low` *float* — Lower end of the supporting price interval.

`price_high` *float* — Upper end of the supporting price interval.

`price` *float* — The price chosen from the interval by `price_rule`.

`price_rule` *EquilibriumPriceRule* — `MIDPOINT` (default), `LOWER` or `UPPER`; the strings `"midpoint"`, `"lower"` and `"upper"` work too.

`traded_values` *tuple* — Values of the traded units.

`traded_costs` *tuple* — Costs of the traded units.

`gains_from_trade` *tuple* — Value minus cost of each traded unit.

`is_unique_price` *bool* — Whether the interval is a single price.

`solve_discrete_market(demand_values, supply_values)` builds both schedules from plain tuples and solves in one call. The demand values must descend and the supply values ascend so that units pair up in order.

```
from principle_viz import (
    DiscreteDemand, DiscreteSupply, solve_discrete_equilibrium,
)

demand = DiscreteDemand((11, 9, 7, 5, 3))
supply = DiscreteSupply((1, 3, 5, 8, 10))
eq = solve_discrete_equilibrium(demand, supply)
print(eq.q_star, eq.price_low, eq.price_high, eq.price)
# 3 5.0 7.0 6.0
print(eq.gains_from_trade)
# (10.0, 6.0, 2.0)
```

Three units trade at a price interval of 5 to 7, and the midpoint rule reports 6. The gains from trade total $10 + 6 + 2 = 18$.

`compute_discrete_surplus` — `compute_discrete_surplus(⟨result⟩)`

Consumer and producer surplus at the chosen price, in total and unit by unit (`consumer_surplus_by_unit`, `producer_surplus_by_unit`). In the example above, both are $5 + 3 + 1 = 9$ at a price of 6.

5.3 Figure

`MarketFigure.add_discrete_curves` — `add_discrete_curves(demand=None, supply=None, *, demand_label="$D$", supply_label="$S$")`

`MarketFigure.add_discrete_equilibrium` — `add_discrete_equilibrium(⟨result⟩)`

Draw each unit as a step $[q, q+1)$: a filled point where the step starts (included) and an open point where it ends (excluded), joined to the next step by a dashed riser. Pass one schedule or both. The equilibrium marks Q^* and the price interval on the axes.

```
fig = MarketFigure(
    x_max=5.5, y_max=12, title="Discrete Demand and Supply",
)
fig.add_discrete_curves(demand, supply)
fig.add_discrete_equilibrium(eq)
fig.finalize()
```

The figure for this example is shown in Figure 4.

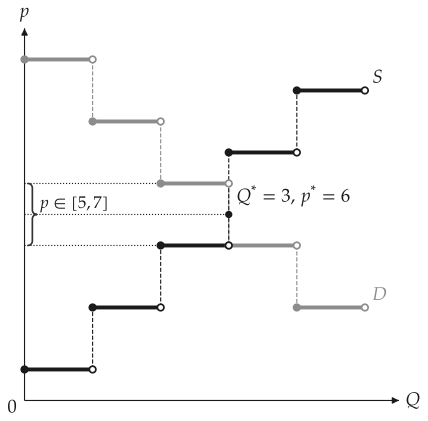


Figure 4
Discrete demand and supply

6 Market curves from individuals

A market curve is the horizontal sum of the individual curves: at each price, market quantity is the sum of the quantities every buyer (or seller) chooses at that price. A buyer buys nothing above the choke price and a seller sells nothing below the minimum price. The market curve of linear individuals is piecewise linear, with a kink where each additional individual enters.

6.1 Summing linear curves

`market_demand` `market_demand(⟨individuals⟩)`
`market_supply` `market_supply(⟨individuals⟩, *, p_max)`

Sum downward-sloping (demand) or upward-sloping (supply) individual lines. The supply sum runs from the lowest minimum price up to `p_max`. Curves of the wrong slope raise `AggregationError`.

`PiecewiseLinear` `PiecewiseLinear(⟨points⟩)`

A demand or supply curve through `points` (Q, p), ordered by quantity. If the first point has $Q = 0$, quantity is zero at prices beyond it.

`q_at(p)` Quantity at a price; a price outside the curve raises `PiecewiseLinearError`.
`p_at(q)` Price at a quantity; a quantity outside the curve raises `PiecewiseLinearError`.
`points` All vertices.
`kinks` The interior vertices where the slope changes.
`domain` Quantity range covered.
`price_range` Price range covered.
`breakpoints()` Vertex prices in ascending order.
`integrate_q()` Area under $Q(p)$ between the prices `p_low` and `p_high`.

```
from principle_viz import (
    line_from_inverse, market_demand, market_supply,
)

a = line_from_inverse(10, -2)     # p = 10 - 2Q
b = line_from_inverse(6, -0.5)    # p = 6 - 0.5Q
demand = market_demand((a, b))
print(demand.points)   # ((0.0, 10.0), (2.0, 6.0), (17.0, 0.0))
print(demand.q_at(4))  # 7.0 = Q_A + Q_B = 3 + 4
```

Buyer B enters at a price of 6, giving the kink $(2, 6)$. At a price of 4, $Q_A = 3$ and $Q_B = 4$, so market demand is 7.

`solve_piecewise_equilibrium`
`piecewise_surplus`

`solve_piecewise_equilibrium(⟨demand⟩, ⟨supply⟩)`
`piecewise_surplus(⟨demand⟩, ⟨supply⟩, ⟨result⟩)`

The exact market equilibrium, solved segment by segment, and the consumer and producer surplus at it, integrated along price.

```
from principle_viz import (
    piecewise_surplus, solve_piecewise_equilibrium,
)

c = line_from_inverse(2, 1)
d = line_from_inverse(5, 0.5)
supply = market_supply((c, d), p_max=10)
eq = solve_piecewise_equilibrium(demand, supply)
print(round(eq.q_star, 3), round(eq.p_star, 3))  # 3.818 5.273
cs, ps = piecewise_surplus(demand, supply, eq)
```

6.2 Summing discrete schedules

`DiscreteDemand.combine(*schedules)` merges every buyer's reservation prices into one market schedule, highest first; `DiscreteSupply.combine()` merges unit costs, lowest first. Equal values remain separate units, and the result works with `solve_discrete_equilibrium()` (Section 5). The output is shown in Figure 6.

```
from principle_viz import DiscreteDemand

first = DiscreteDemand((10, 7, 4))
second = DiscreteDemand((8, 5, 2))
market = DiscreteDemand.combine(first, second)
print(market.values)
# (10.0, 8.0, 7.0, 5.0, 4.0, 2.0)
```

6.3 Aggregation figures

`demand_aggregation_figure`
`supply_aggregation_figure`
`discrete_demand_aggregation_figure`
`discrete_supply_aggregation_figure`

`demand_aggregation_figure(⟨individuals⟩, *, price, price_label="$p_1$", link_price=False, ...)`
`supply_aggregation_figure(⟨individuals⟩, *, price, p_max, ...)`
`discrete_demand_aggregation_figure(⟨individuals⟩, *, price, ...)`
`discrete_supply_aggregation_figure(⟨individuals⟩, *, price, ...)`

New: v0.10.0

One panel per individual, then the market, side by side and sharing the price axis. `individuals` maps each name to a curve or schedule; the curves are named D_A, D_B, ... and D (or S_A, ..., S). Dashed guides at `price` mark Q_A, Q_B and $Q_A + Q_B = Q$. `supply_aggregation_figure()` also needs `p_max`.

`link_price` — *bool* · default `False`
Run the price line across every panel and the gaps between them, marking the price on the first panel only.

`theme`
As for `MarketFigure` (Section 15.3).

`palette`
As for `MarketFigure` (Section 15.3).

`labels`
Label overrides by layer id (Section 15.2).

`visibility`
Layer visibility by layer id (Section 15.2).

The returned `AggregationFigure` has `save()`, `hide()`, `show()`, `configure_label()`, `layer_ids` and `label_ids`, like `MarketFigure`.

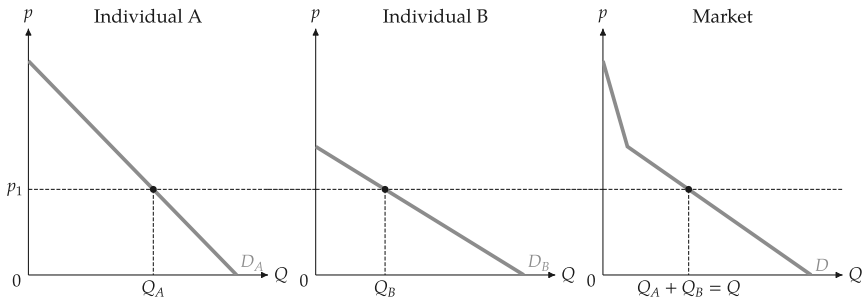


Figure 5
Market demand as horizontal sum

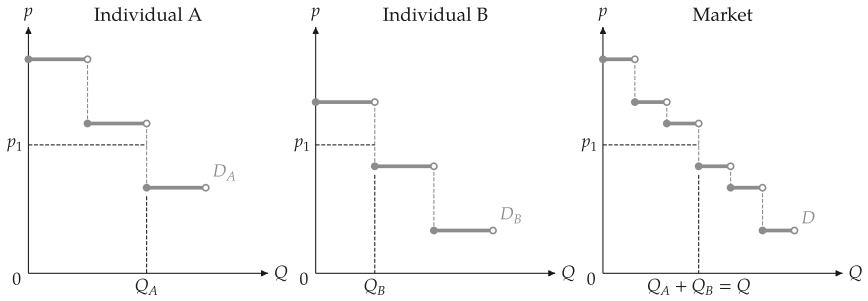


Figure 6
Combining two discrete demands

```
from principle_viz import demand_aggregation_figure

fig = demand_aggregation_figure(
    {"A": a, "B": b}, price=4, link_price=True,
)
fig.save("market_demand.png")
```

The output of `demand_aggregation_figure()` is shown in Figure 5.

7 Elasticity and total revenue

7.1 Price elasticity

`compute_point_elasticity`
`compute_arc_elasticity`

`compute_point_elasticity(⟨demand⟩, ⟨quantity⟩)`
`compute_arc_elasticity(q0, p0, q1, p1)`

The point price elasticity of a line at a quantity,

$$\varepsilon = \frac{\mathrm{d}Q}{\mathrm{d}p} \cdot \frac{p}{Q},$$

and the arc (midpoint) elasticity between two points,

$$\varepsilon = \frac{\Delta Q / \overline{Q}}{\Delta p / \overline{p}},$$

where $\overline{Q}$ and $\overline{p}$ are the averages of the two points. Both are negative for a downward-sloping demand.

```
from principle_viz import (
    compute_arc_elasticity, compute_point_elasticity, line_from_inverse,
)

demand = line_from_inverse(10.0, -1.0)
print(compute_point_elasticity(demand, 4))   # -1.5
print(compute_point_elasticity(demand, 5))   # -1.0 (unit elastic)
print(compute_arc_elasticity(4, 6, 6, 4))    # -1.0
```

At $Q = 4$ the elasticity is -1.5, so demand is elastic. $Q = 5$ is unit elastic, the midpoint of $p = 10 - Q$. The arc elasticity uses the averages of the two points, so $(4, 6)$ to $(6, 4)$ and the reverse give the same value.

`classify_elasticity(value)` in `principle_viz.core.elasticity` names the absolute value: `"elastic"` above 1, `"unit_elastic"` at 1 and `"inelastic"` below 1.

7.2 Total revenue

`elasticity_revenue_schedule`

`elasticity_revenue_schedule(⟨demand⟩, *, samples=101)`

Sample a linear demand from $Q = 0$ to its choke quantity and return, at each point, price, total revenue pQ, elasticity and its class. Total revenue peaks where demand is unit elastic, at the midpoint of the line.

`points` `RevenuePoints` with `quantity`, `price`, `total_revenue`, `elasticity` and `classification`.

`unit_elastic_quantity` Quantity at the midpoint of the line.

`unit_elastic_price` Price at the midpoint of the line.

`maximum_revenue` Total revenue at that point.

`choke_quantity` The quantity-axis intercept.

`choke_price` The price-axis intercept.

For $p = 12 - Q$, maximum revenue is 36 at $Q = 6$, $p = 6$.

`elasticity_revenue_canvases`

`elasticity_revenue_canvases(⟨demand⟩, ⟨result⟩, *, theme=None, labels=None, visibility=None)`

Two mosaickit canvases, in `principle_viz.visuals.revenue`: the demand curve, labelled "Elastic", "Unit elastic" and "Inelastic" along its length with the unit-elastic point marked, and total revenue against quantity with its maximum marked. Each canvas carries its own title. Place them side by side with `mosaickit.CanvasGrid` (see Figure 7).

```
from mosaickit import CanvasGrid
from principle_viz import PlotTheme, elasticity_revenue_schedule
from principle_viz.visuals.revenue import (
    elasticity_revenue_canvases,
)

demand = line_from_inverse(12.0, -1.0)
schedule = elasticity_revenue_schedule(demand)
canvases = elasticity_revenue_canvases(
    demand, schedule, theme=PlotTheme(),
)
CanvasGrid(canvases, rows=1).save("elasticity_total_revenue.png")
```

8 Welfare

Welfare analysis compares a market outcome with its alternatives: consumer surplus, producer surplus, government revenue and deadweight loss.

8.1 Market outcomes

`MarketOutcome`

`MarketOutcome(quantity, consumer_price, producer_price, label="")`

A quantity with the price buyers pay and the price sellers receive. The two prices differ under a tax or a subsidy; the gap times the quantity is government revenue (negative for a subsidy). It lives in `principle_viz.welfare.surplus`, together with constructors from each kind of result:

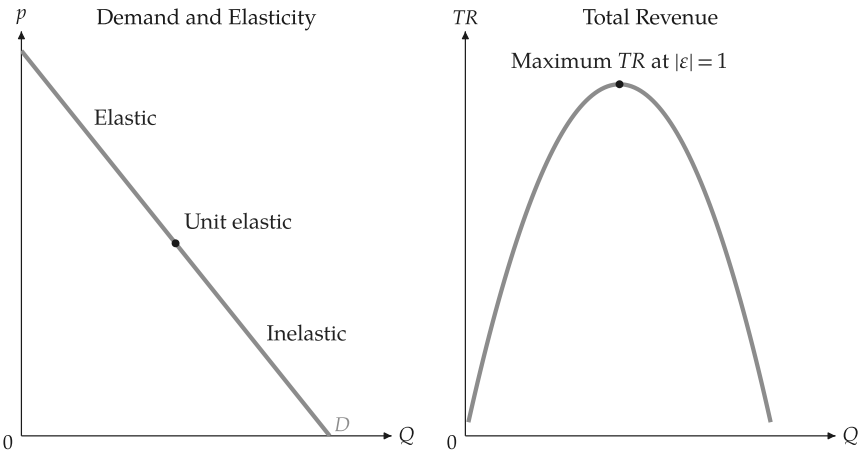


Figure 7
Elasticity and total revenue

`outcome_from_equilibrium(eq)` A free market: both prices are p^*.

`outcome_from_tax(tax_eq)` After a tax (Section 9).

`outcome_from_subsidy(subsidy_eq)` After a subsidy.

`outcome_from_control(control)` Under a price ceiling or floor (Section 10).

8.2 Surplus

`compute_surplus` `compute_surplus(⟨demand⟩, ⟨supply⟩, ⟨outcome⟩, baseline_outcome=None)`

The welfare decomposition at `outcome`. With a `baseline_outcome`, the deadweight loss is measured against it. The `SurplusResult` has these fields:

`consumer_surplus` *float* Area between demand and the consumer price.

`producer_surplus` *float* Area between the producer price and supply.

`tax_revenue` *float* Government revenue (negative for a subsidy).

`total_surplus` *float* The sum of the three.

`deadweight_loss` *float* Surplus lost relative to the baseline.

`polygons` *SurplusPolygons* The corner points of each area, used to draw them.

```
from principle_viz import compute_surplus, solve_equilibrium
from principle_viz.welfare.surplus import outcome_from_equilibrium

eq = solve_equilibrium(demand, supply)
outcome = outcome_from_equilibrium(eq)
surplus = compute_surplus(demand, supply, outcome)
print(surplus.consumer_surplus, surplus.producer_surplus)  # 8.0 8.0
```

$(10 - 6) \times 4/2 = (6 - 2) \times 4/2 = 8$; the free market has no deadweight loss.

`compare_surplus` `compare_surplus(⟨demand⟩, ⟨supply⟩, baseline_outcome, policy_outcome)`

Both decompositions and the changes between them: `baseline`, `policy`, `delta_consumer_surplus`, `delta_producer_surplus`, `delta_tax_revenue`, `delta_total_surplus` and `deadweight_loss`. Section 9 uses it for a tax.

8.3 Figures

`MarketFigure.add_welfare`

`add_welfare(`⟨*result*⟩`, *, labels=True, regions=None)`

Shade consumer surplus, producer surplus, tax revenue and deadweight loss, and name each region: inside it when the name fits, by its short name (CS, PS, Tax, DWL) when only that fits, otherwise by a callout that covers no line, point or other text.

`labels`
bool · default `True`

`False` shades the regions without naming them.

`regions`
iterable | *None* · default `None`

Limit the shading to some of `"cs"`, `"ps"`, `"tax_revenue"` and `"dwl"`.

```
fig = MarketFigure(x_max=12, y_max=12)
fig.add_curves(demand, supply, q_max=10)
fig.add_welfare(surplus)
fig.add_equilibrium(eq)
fig.finalize()
```

The output is shown in Figure 8.

`MarketFigure.add_welfare_transition`

`add_welfare_transition(*, baseline_outcome, policy_outcome, surplus)`

The same regions together with guide lines at the baseline and policy quantities and prices, for comparing before and after.

8.4 Deadweight-loss reports

`build_dwl_report`

`build_dwl_report(`⟨*rows*⟩`)`

Turn `(name, baseline, policy)` triples of `SurplusResults` into report rows (`DWLScenarioRow`) with the change in quantity, in each surplus and the deadweight loss. `save_dwl_report_csv()` and `save_dwl_report_json()` in `principle_viz.welfare.report` write them to a file.

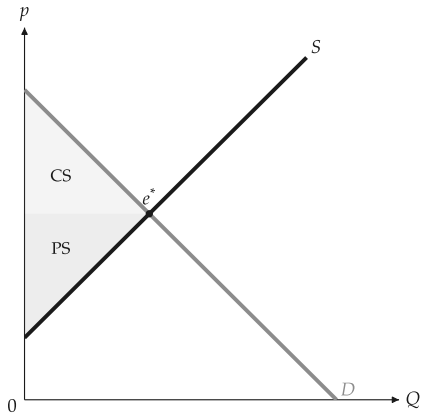


Figure 8
Surplus at equilibrium

9 Taxes and subsidies

A tax drives a wedge between the price buyers pay and the price sellers receive. The wedge, the quantity and both prices are the same whether the tax is charged to buyers or to sellers (Mankiw, 2021).

9.1 Scenarios

`TaxScenario` `TaxScenario(tax_type, amount, tax_on=TaxOn.PRODUCER, anchor_mode=AnchorMode.NONE)`

One tax, from `principle_viz.policy.tax`.

`tax_type` *TaxType*
`FIXED_TAX` or `PER_UNIT_TAX`: an amount per unit, which shifts a curve by `amount`; `AD_VALOREM_TAX`: a rate on the price, which rotates a curve. The two specific taxes give the same equilibrium.

`amount` *float*
The tax per unit, or the rate for an ad valorem tax (`0.35` is 35%); a rate must exceed −1.

`tax_on` *TaxOn* · default `PRODUCER`
Legal incidence: `PRODUCER` shifts supply up, `CONSUMER` shifts demand down.

`anchor_mode` *AnchorMode* · default `NONE`
For drawing an ad valorem tax: `NONE` rotates the curve about its intercept, `BASELINE_EQUILIBRIUM` about the untaxed equilibrium.

9.2 Solving

`TaxType` `TaxType.FIXED_TAX`, `PER_UNIT_TAX`, `AD_VALOREM_TAX`

The tax's definition, from `principle_viz.policy.tax`.

`FIXED_TAX` *TaxType*
A fixed amount, which shifts a curve by `amount`.

`PER_UNIT_TAX` *TaxType*
An amount per unit; the same equilibrium as `FIXED_TAX`.

`AD_VALOREM_TAX` *TaxType*
A rate on the price, which rotates a curve.

`TaxOn` `TaxOn.CONSUMER`, `PRODUCER`

The legal incidence of a tax, from `principle_viz.policy.tax`.

`CONSUMER` *TaxOn*
Charged to buyers: demand shifts down.

`PRODUCER` *TaxOn*
Charged to sellers: supply shifts up.

`solve_tax_equilibrium` `solve_tax_equilibrium(⟨demand⟩, ⟨supply⟩, ⟨scenario⟩)`

The taxed market, as a `TaxEquilibriumResult`: `q_star`, `consumer_price`, `producer_price`, `tax_wedge` (their difference) and `tax_revenue` (wedge times quantity).

```
from principle_viz import solve_tax_equilibrium
from principle_viz.policy.tax import TaxOn, TaxScenario, TaxType

tax = TaxScenario(TaxType.PER_UNIT_TAX, 3.0, TaxOn.PRODUCER)
print(solve_tax_equilibrium(demand, supply, tax))
# TaxEquilibriumResult(q_star=2.5, consumer_price=7.5,
#                      producer_price=4.5, tax_wedge=3.0,
#                      tax_revenue=7.5)
```

`TaxOn.CONSUMER` gives the same result. An ad valorem tax at rate r sets the consumer price to $(1 + r)$ times the producer price.

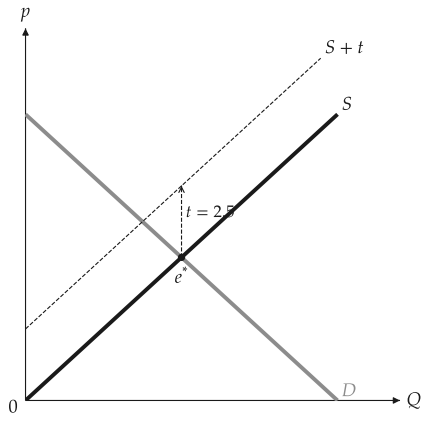


Figure 9
Per-unit tax on sellers

`compare_tax_scenario` `compare_tax_scenario(⟨demand⟩, ⟨supply⟩, ⟨scenario⟩)`

The untaxed and taxed markets side by side: `baseline_equilibrium`, `post_tax`, the changes `delta_q`, `delta_p_consumer`, `delta_p_producer`, and their directions (`"left"`/`"right"`, `"up"`/`"down"`).

`build_tax_visual_guide` `build_tax_visual_guide(⟨demand⟩, ⟨supply⟩, ⟨scenario⟩)`

The geometry of the taxed curve: `base_curve`, `taxed_curve`, `curve_role` (`"supply"` or `"demand"`) and `transform_kind` (`"shift"` or `"rotation"`). `add_tax_transform()` draws from it.

9.3 Figures

`MarketFigure.add_tax_transform` `add_tax_transform(⟨demand⟩, ⟨supply⟩, ⟨scenario⟩, q_max)`

Draw the taxed curve, named $S + t$ or $D - t$, with a dashed arrow for the shift or the rotation and the untaxed equilibrium (see Figure 9 and Figure 10).

`MarketFigure.add_tax_comparison` `add_tax_comparison(⟨result⟩, *, brace_side="outside", notes=False)`

Mark the wedge of a `compare_tax_scenario()` result: p_d (buyers), p_0 (before the tax) and p_s (sellers) on the price axis, with a "Tax" brace over p_s to p_d.

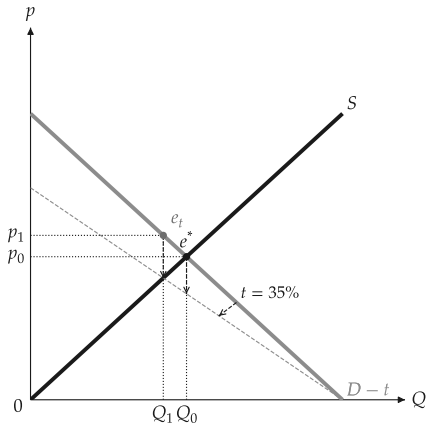


Figure 10
Ad valorem tax on buyers

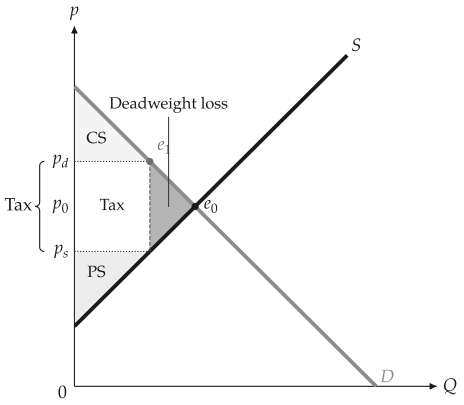


Figure 11
Surplus and loss under a tax

`brace_side`
str · default `"outside"`
`"outside"` the axis or `"inside"` the plot.

`notes`
bool · default `False`
Explain each mark beside the axis.

It combines with the welfare regions of Section 8 (see Figure 11):

```
from principle_viz import compare_tax_scenario
from principle_viz.welfare.surplus import (
    compare_surplus, outcome_from_equilibrium, outcome_from_tax,
)

eq = solve_equilibrium(demand, supply)
baseline = outcome_from_equilibrium(eq)
taxed = outcome_from_tax(solve_tax_equilibrium(demand, supply, tax))
delta = compare_surplus(demand, supply, baseline, taxed)
print(delta.policy.tax_revenue, delta.deadweight_loss)  # 7.5 2.25

fig = MarketFigure(x_max=12, y_max=12, title="Welfare Under a Tax")
fig.add_curves(demand, supply, q_max=10)
fig.add_welfare(delta.policy)
fig.add_tax_comparison(compare_tax_scenario(demand, supply, tax))
fig.finalize()
```

9.4 Subsidies

`SubsidyScenario`
`SubsidyScenario(amount, subsidy_to=SubsidyTo.PRODUCER)`
A per-unit subsidy of `amount` (non-negative), paid to sellers (`SubsidyTo.PRODUCER`) or buyers (`SubsidyTo.CONSUMER`), from `principle_viz.policy.subsidy`.

`SubsidyTo`
`SubsidyTo.PRODUCER`, `CONSUMER`
Who receives a subsidy, from `principle_viz.policy.subsidy`.

`PRODUCER`
SubsidyTo
Paid to sellers.

`CONSUMER`
SubsidyTo
Paid to buyers.

`solve_subsidy_equilibrium`
`compare_subsidy_scenario`
`solve_subsidy_equilibrium(⟨demand⟩, ⟨supply⟩, ⟨scenario⟩)`
`compare_subsidy_scenario(⟨demand⟩, ⟨supply⟩, ⟨scenario⟩)`

The subsidised market (`q_star`, `consumer_price`, `producer_price`, `subsidy_wedge`, `government_expenditure`), and its comparison with the free market (`baseline_equilibrium`, `post_subsidy`, `delta_q`, `delta_p_consumer`, `delta_p_producer`). A subsidy

of 2 in the market of Section 3 gives a quantity of 5; buyers pay 5, sellers receive 7 and government expenditure is 10.

`MarketFigure.add_subsidy_comparison`

`add_subsidy_comparison(⟨result⟩, *, brace_side="outside", notes=False)`

Mark the subsidy wedge like the tax wedge, with a "Subsidy" brace, and name the subsidy's cost. Figure 12 shades only the deadweight loss (`regions=("dwl",)`) and hides some labels with `visibility` (Section 15.2).

```
from principle_viz import (
    SubsidyScenario, SubsidyTo, compare_subsidy_scenario,
)

demand = line_from_inverse(12.0, -1.0)
subsidy = SubsidyScenario(3.0, SubsidyTo.PRODUCER)
comparison = compare_subsidy_scenario(demand, supply, subsidy)
fig = MarketFigure(
    x_max=12, y_max=13, title="Per-Unit Subsidy",
    visibility={
        "market.subsidy.expenditure.label": False,
        "market.subsidy.wedge.label": False,
        "market.subsidy.wedge.mark.p_0": False,
        "market.subsidy.wedge.brace": False,
    },
)
```

10 Price controls

A price ceiling below the equilibrium price, or a floor above it, binds: the market does not clear, and the quantity traded is the smaller of the quantities demanded and supplied at the controlled price.

`PriceControlScenario` `PriceControlScenario(control_type, control_price)`

`PriceControlType` `PriceControlType.CEILING | PriceControlType.FLOOR`

A `CEILING` or `FLOOR` at `control_price`, from `principle_viz.core.controls`.

`CEILING` *PriceControlType* — A maximum price.

`FLOOR` *PriceControlType* — A minimum price.

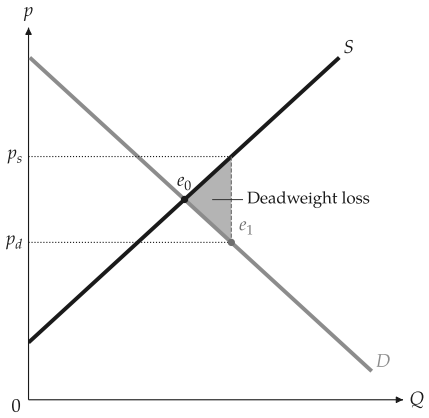


Figure 12
Per-unit subsidy and its loss

`evaluate_price_control` `evaluate_price_control(⟨demand⟩, ⟨supply⟩, ⟨scenario⟩)`

The controlled market, as a `PriceControlResult`:

`is_binding` *bool* — Whether the control changes the outcome.

`traded_quantity` *float* — Quantity actually traded.

`consumer_price` *float* — Price buyers pay: the controlled price when binding, otherwise the equilibrium price.

`producer_price` *float* — Price sellers receive: the controlled price when binding, otherwise the equilibrium price.

`shortage` *float* — Excess demand under a ceiling (zero otherwise).

`surplus` *float* — Excess supply under a floor (zero otherwise).

`baseline_equilibrium` *EquilibriumResult* — The uncontrolled market.

```
from principle_viz import evaluate_price_control
from principle_viz.core.controls import (
    PriceControlScenario, PriceControlType,
)

scenario = PriceControlScenario(PriceControlType.CEILING, 4.0)
ceiling = evaluate_price_control(demand, supply, scenario)
print(ceiling.is_binding, ceiling.traded_quantity, ceiling.shortage)
# True 2.0 4.0
```

A ceiling above the equilibrium price (for example 7) leaves `is_binding` `False`.

`MarketFigure.add_price_control` `add_price_control(⟨result⟩, *, gap_brace="line")`

Draw the control line, named “Price ceiling” or “Price floor”, with p_c on the price axis. A binding control also marks Q_d and Q_s and braces the gap: “Shortage” below a ceiling, “Surplus” above a floor. `gap_brace="axis"` braces it under the quantity axis instead (see Figure 13 and Figure 14).

`outcome_from_control()` turns the result into a market outcome for the welfare functions of Section 8 (see Figure 15):

```
from principle_viz.welfare.surplus import (
    compare_surplus, outcome_from_control, outcome_from_equilibrium,
)
```

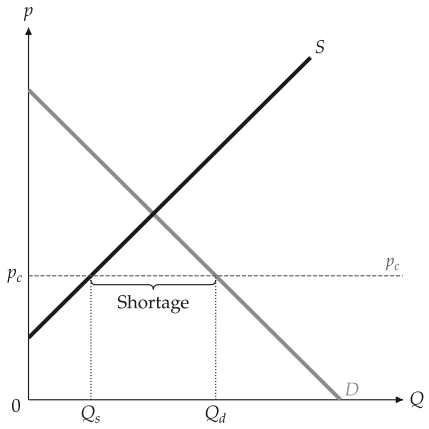


Figure 13
Binding price ceiling

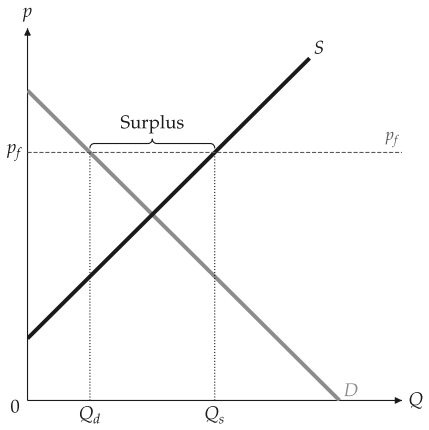


Figure 14
Binding price floor

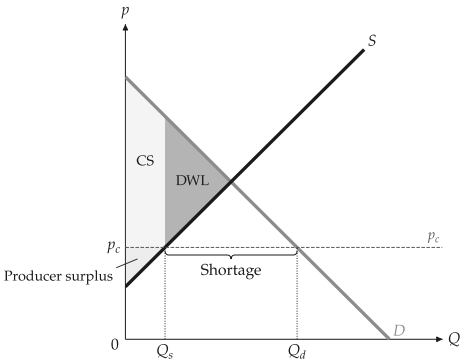


Figure 15
Welfare under a ceiling of 3.5

```
eq = solve_equilibrium(demand, supply)
baseline = outcome_from_equilibrium(eq)
controlled = outcome_from_control(ceiling)
delta = compare_surplus(demand, supply, baseline, controlled)
fig.add_price_control(ceiling)
fig.add_welfare(delta.policy)
```

Under a price ceiling, the deadweight loss is the surplus of the units that are no longer traded, and part of the remaining surplus transfers from sellers to buyers.

11 International trade

A small open economy takes the world price p_w as given. Below the autarky price it imports the gap between domestic demand and supply; above it, it exports. A tariff or an import quota raises the domestic price and shrinks imports.

`TradeScenario` `TradeScenario(world_price, tariff=0.0, import_quota=None, quota_rent_recipient=QuotaRentRecipient.DOMESTIC)`

The world price and at most one policy: a per-unit `tariff` or an `import_quota`, not both (`PolicyError`).

`quota_rent_recipient` *QuotaRentRecipient*

Who gains the quota rent: `DOMESTIC` importers or `GOVERNMENT` (both count in national surplus; the government's share is reported as revenue), or `FOREIGN` exporters (it leaves the country).

`analyze_trade` `analyze_trade(`⟨*demand*⟩`,` ⟨*supply*⟩`,` ⟨*scenario*⟩`)`

Compare autarky, free trade and the policy. The `TradeComparisonResult` holds three `TradeOutcomes`, `autarky`, `free_trade` and `policy`, and the `deadweight_loss` of the policy relative to free trade.

`domestic_price` Price in the home market.

`quantity_demanded` Domestic quantity demanded at that price.

`quantity_supplied` Domestic quantity supplied at that price.

`imports` Quantity imported.

`exports` Quantity exported.

`direction` `TradeDirection`: `IMPORT`, `EXPORT` or `AUTARKY`.

`consumer_surplus` Consumer surplus.

`producer_surplus` Producer surplus.

`government_revenue` Government revenue.

`quota_rent` Rent created by a quota.

`national_quota_rent` The part of the quota rent that stays in the country.

`total_surplus` National surplus.

`gains_from_trade` Gain over autarky.

`is_policy_binding` Whether the tariff or quota changes the price.

```
from principle_viz import TradeScenario, analyze_trade

demand = line_from_inverse(12.0, -1.0)   # autarky price 7
scenario = TradeScenario(world_price=4, tariff=2)
result = analyze_trade(demand, supply, scenario)
print(result.free_trade.imports, result.policy.imports)
# 6.0 2.0
print(result.policy.government_revenue, result.deadweight_loss)
# 4.0 4.0
```

With the tariff of 2, the domestic price is 6 and government revenue is $2 \times 2 = 4$. An import quota of 2 units gives the same price; the rent of 4 goes to the party named by `quota_rent_recipient`.

`MarketFigure.add_trade` `add_trade(⟨result⟩)`

Draw the world price line, named p_w, the policy price ($p_w + t$ or p_q), Q_s and Q_d on the quantity axis with an "Imports" or "Exports" brace beneath, and the tariff revenue or quota rent as a named rectangle (see Figure 16, Figure 17 and Figure 18).

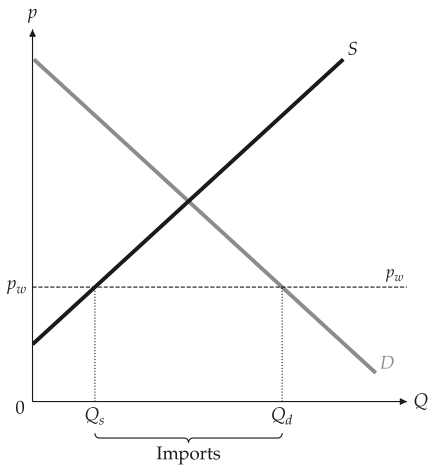


Figure 16
Free trade with imports

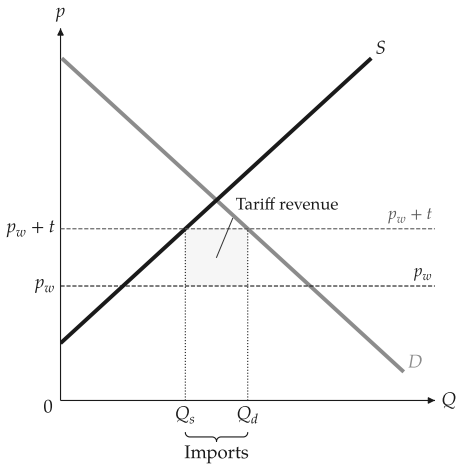


Figure 17
Import tariff

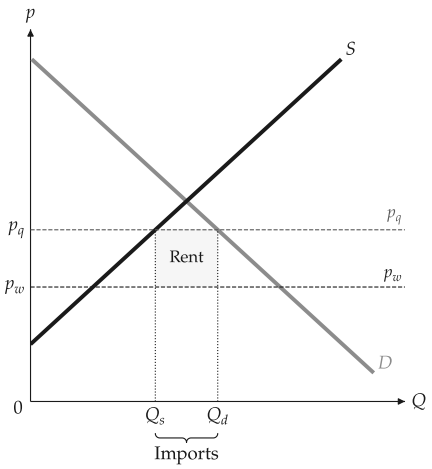


Figure 18
Binding import quota

12 Market failures

12.1 Externalities

When production imposes a cost on third parties, or consumption confers a benefit on them, the market quantity differs from the social optimum. A Pigouvian tax or subsidy equal to the external effect at the optimum restores the optimal quantity (Pigou, 1920).

`ExternalityScenario` `ExternalityScenario(marginal_external_cost=0.0, marginal_external_benefit=0.0)`

A constant marginal external cost, which adds to supply to give the marginal social cost, and/or a marginal external benefit, which adds to demand to give the marginal social benefit. Both must be non-negative.

`analyze_externality` `analyze_externality(⟨demand⟩, ⟨supply⟩, ⟨scenario⟩)`

The private and social outcomes: `private_equilibrium`, `social_equilibrium`, `social_demand`, `social_supply`, `corrective_tax`, `corrective_subsidy`, `quantity_distortion` and `deadweight_loss`.

```
from principle_viz import ExternalityScenario, analyze_externality

demand = line_from_inverse(12.0, -1.0)
scenario = ExternalityScenario(marginal_external_cost=2)
result = analyze_externality(demand, supply, scenario)
```

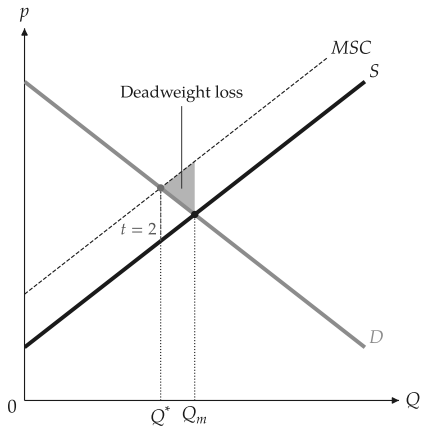


Figure 19
Negative externality

```
print(result.private_equilibrium.q_star, result.social_equilibrium.q_star)
# 5.0 4.0
print(result.corrective_tax, result.deadweight_loss)
# 2.0 1.0
```

`MarketFigure.add_externality`

`add_externality(⟨result⟩)`

Draw the social curve, mark Q_m (market) and Q^* (optimum) on the quantity axis, label the corrective tax t or subsidy s across the gap at Q^*, and shade the deadweight loss (see Figure 19 and Figure 20).

12.2 Common resources

`analyze_common_resource`

`analyze_common_resource(⟨benefit⟩, ⟨cost⟩, *, marginal_congestion_cost)`

Treat congestion or depletion as a marginal external cost: open access uses the resource until marginal benefit equals private cost, beyond the efficient level (Hardin, 1968). The result has `open_access_equilibrium`, `efficient_equilibrium`, `social_cost`, `overuse`, `corrective_fee` and `deadweight_loss`.

With $MB = 12 - Q$, $MPC = 2 + Q$ and a congestion cost of 3, open access uses 5 units, the efficient level is 3.5 and `corrective_fee` is 3.

`MarketFigure.add_common_resource`

`add_common_resource(⟨result⟩)`

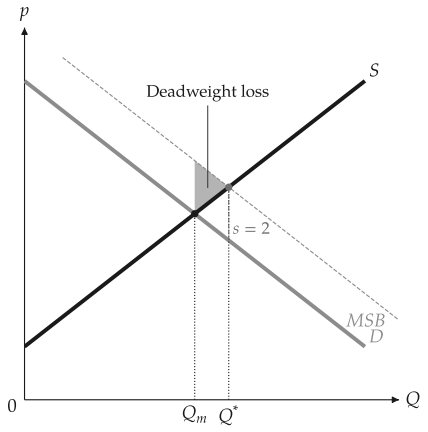


Figure 20
Positive externality

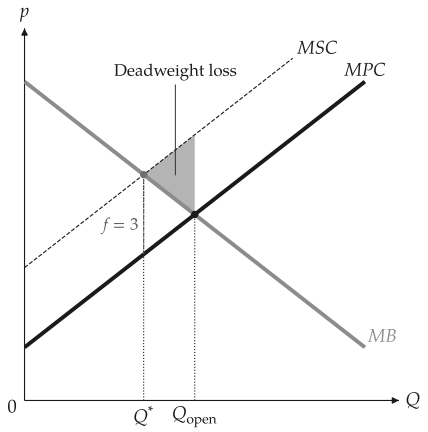


Figure 21
Overuse of a common resource

Draw the social cost, mark Q^* and Q_{open}, and shade the deadweight loss. Name the curves *MB* and *MPC* in `add_curves()`. See Figure 21.

12.3 Public goods

`IndividualBenefit` `IndividualBenefit(name, marginal_benefit)`

`analyze_public_good` `analyze_public_good(⟨individuals⟩, ⟨cost⟩, *, samples=101)`

Everyone consumes the whole quantity of a public good, so marginal benefits add vertically. The efficient quantity sets their sum equal to marginal cost (Samuelson, 1954); private provision stops where the highest individual benefit meets marginal cost. The result has `efficient_quantity`, `efficient_marginal_value`, `private_provision_quantity`, `free_rider_gap` and the sampled `points`.

```
from principle_viz import IndividualBenefit, analyze_public_good

result = analyze_public_good(
    (
        IndividualBenefit("$MB_A$", line_from_inverse(8, -1)),
        IndividualBenefit("$MB_B$", line_from_inverse(6, -1)),
    ),
    line_from_inverse(5, 0),  # constant marginal cost of 5
)
print(result.efficient_quantity, result.private_provision_quantity)
# 4.5 3.0
```

`public_good_canvas` `public_good_canvas(⟨result⟩, *, theme=None, labels=None, visibility=None)`

A mosaickit canvas, in `principle_viz.visuals.market_failures`, with each marginal benefit, their vertical sum, marginal cost, and Q_p and Q^* on the quantity axis (see Figure 22).

13 Labor and loanable funds

13.1 Minimum wage

`analyze_minimum_wage` `analyze_minimum_wage(⟨demand⟩, ⟨supply⟩, minimum_wage)`

A competitive labor market with a wage floor. The `MinimumWageResult` has `equilibrium`, `minimum_wage`, `is_binding`, `labor_demanded`, `labor_supplied`, `employment` (the smaller of the two), `unemployment` (their gap) and `wage_bill`.

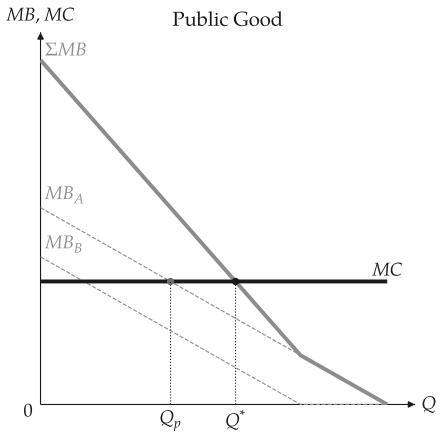


Figure 22
Vertical sum of marginal benefits

```
from principle_viz import analyze_minimum_wage

labor_demand = line_from_inverse(12, -1)
labor_supply = line_from_inverse(2, 1)
labor = analyze_minimum_wage(
    labor_demand, labor_supply, minimum_wage=9,
)
print(labor.employment, labor.unemployment)  # 3.0 4.0
```

`MarketFigure.add_minimum_wage`

`add_minimum_wage(⟨result⟩, *, gap_brace="line")`

The wage floor drawn like a price floor (Section 10): the "Minimum wage" line, $w_{\min}$ on the wage axis, and for a binding floor L_d and L_s with an "Unemployment" brace. Title the axes L and w and name the curves D_L and S_L (see Figure 23):

```
fig = MarketFigure(x_max=11, y_max=14, x_label="L", y_label="w")
fig.add_curves(
    labor_demand, labor_supply, q_max=10,
    demand_label="$D_L$", supply_label="$S_L$",
)
fig.add_minimum_wage(labor)
```

`labor_demanded` is 3, `labor_supplied` is 7, `employment` is 3 and `unemployment` is 4.

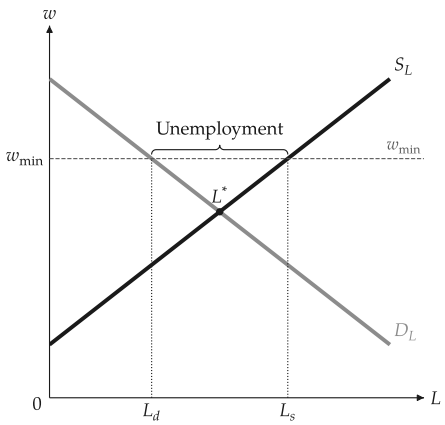


Figure 23
Binding minimum wage

13.2 Loanable funds

`LoanableFundsScenario` `LoanableFundsScenario(savings_quantity_shift=0.0, investment_quantity_shift=0.0, government_borrowing=0.0)`

Horizontal shifts of saving (supply) and investment (demand), measured in quantity. Government borrowing (non-negative) adds to the demand for loanable funds.

`analyze_loanable_funds` `analyze_loanable_funds(⟨savings⟩, ⟨investment⟩, ⟨scenario⟩)`

The market for loanable funds before and after the shift: `baseline_equilibrium`, `shifted_equilibrium`, `shifted_savings`, `shifted_investment_demand`, `private_investment_after`, `interest_rate_change` and `crowding_out`, the private investment displaced by the higher interest rate.

```
from principle_viz import (
    LoanableFundsScenario, analyze_loanable_funds,
)

savings = line_from_inverse(2, 0.5)
investment = line_from_inverse(12, -0.5)
scenario = LoanableFundsScenario(government_borrowing=4)
funds = analyze_loanable_funds(savings, investment, scenario)
print(funds.interest_rate_change, funds.crowding_out)  # 1.0 2.0
```

`interest_rate_change` is 1.0 and `crowding_out` is 2.0.

`MarketFigure.add_loanable_funds` `add_loanable_funds(⟨result⟩)`

Draw the shifted curves, named D_1 or S_1, both equilibria and the movement between them. Title the price axis r (see Figure 24).

14 Production possibilities

`ProductionPossibilitiesFrontier` `ProductionPossibilitiesFrontier(x_intercept, y_intercept, curvature=1.0, x_good="Good X", y_good="Good Y")`

The frontier

$$y = Y(1 - (x/X)^c),$$

through the intercepts X and Y. A curvature $c = 1$ gives a straight line (constant opportunity cost); $c > 1$ bows it out (increasing opportunity cost). Intercepts and curvature must be positive and $c \geq 1$, otherwise `PPFError` is raised.

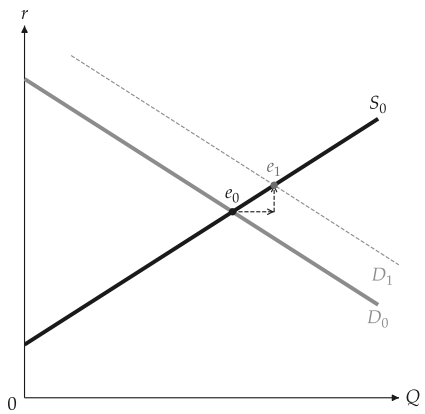


Figure 24 Borrowing crowds out investment

`y_at(x)` Output of y when x units of the first good are produced.

`opportunity_cost_x(x)` Units of y given up for one more unit of x, $|\mathrm{d}y/\mathrm{d}x|$.

`assess(x, y)` A `PointStatus`: `EFFICIENT` on the frontier, `INEFFICIENT` inside it, `UNATTAINABLE` outside.

`PointStatus` `PointStatus.EFFICIENT`, `INEFFICIENT`, `UNATTAINABLE`

Where a point lies relative to the frontier.

`EFFICIENT` *PointStatus* On the frontier.

`INEFFICIENT` *PointStatus* Inside the frontier.

`UNATTAINABLE` *PointStatus* Outside the frontier.

```
from principle_viz import ProductionPossibilitiesFrontier

ppf = ProductionPossibilitiesFrontier(
    10, 8, curvature=2,
    x_good="Consumer goods", y_good="Capital goods",
)
print(ppf.y_at(6), ppf.opportunity_cost_x(6))  # 5.12 0.96
print(ppf.assess(4, 3), ppf.assess(7, 6))
# PointStatus.INEFFICIENT PointStatus.UNATTAINABLE
```

`analyze_ppf` `analyze_ppf(⟨frontier⟩, points=(), *, samples=101)`

`ppf_canvas` `ppf_canvas(⟨result⟩, *, theme=None, labels=None, visibility=None)`

Sample the frontier and assess named points, given as `(x, y, label)` triples; `ppf_canvas()`, in `principle_viz.visuals.ppf`, draws it with the attainable set shaded and the points labelled (Figure 25).

```
from principle_viz import analyze_ppf
from principle_viz.visuals.ppf import ppf_canvas

points = ((6, ppf.y_at(6), "A"), (4, 3, "B"), (7, 6, "C"))
result = analyze_ppf(ppf, points=points)
ppf_canvas(result).save("ppf_points.png")
```

`PPFGrowthScenario` `PPFGrowthScenario(x_growth_rate=0.0, y_growth_rate=0.0)`

`analyze_ppf_growth` `analyze_ppf_growth(⟨frontier⟩, ⟨scenario⟩, *, samples=101)`

`ppf_growth_canvas` `ppf_growth_canvas(⟨result⟩, *, theme=None, labels=None, visibility=None)`

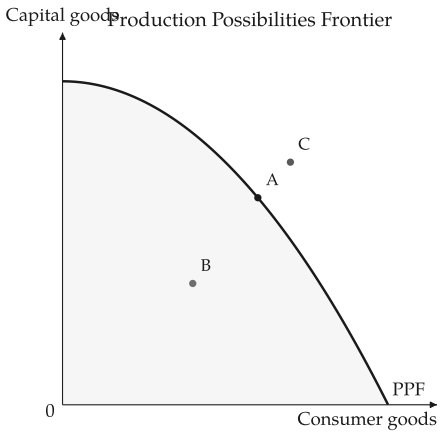


Figure 25
Efficient and inefficient points

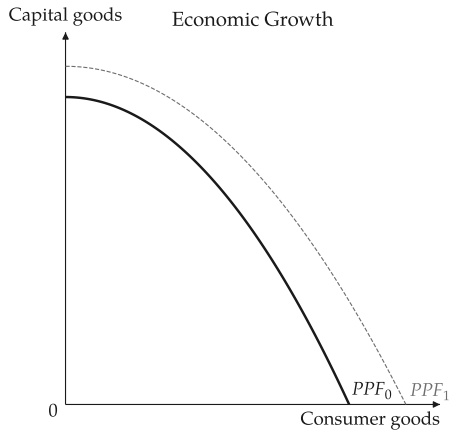


Figure 26
Growth in consumer and capital goods

Economic growth scales each intercept by one plus its growth rate; the result holds the `baseline` and `shifted` frontiers and their sampled points. `ppf_growth_canvas()` draws both, named PPF_0 and PPF_1 (see Figure 26).

14.1 Comparative advantage

`compare_linear_ppfs` `compare_linear_ppfs(name_a, frontier_a, name_b, frontier_b)`

For two straight-line frontiers, the opportunity cost of x for each producer and who has the comparative advantage in each good (`"tie"` when the costs are equal). Curved frontiers raise `PPFError`.

```
from principle_viz import compare_linear_ppfs

ann = ProductionPossibilitiesFrontier(10, 5)  # x costs 0.5 y
bob = ProductionPossibilitiesFrontier(6, 6)   # x costs 1 y
result = compare_linear_ppfs("Ann", ann, "Bob", bob)
print(result.comparative_advantage_x, result.comparative_advantage_y)
# Ann Bob
```

Ann has the comparative advantage in x (0.5 against 1) and Bob in y (1 against 2).

15 Figures

Every figure in this manual is a `MarketFigure`, or a mosaickit canvas built by a function such as `ppf_canvas()`. Both have a square plot area and arrowed axes with no ticks or grid. Curves are named at their ends, areas inside them and values on the axes, and no text covers a line, a point, an area or other text.

15.1 MarketFigure

`MarketFigure`

```
from principle_viz import MarketFigure

fig = MarketFigure(
    x_max=20.0,
    y_max=20.0,
    x_label="Q",
    y_label="P",
    title="Market Diagram",
    theme=None,       # PlotTheme
    palette=None,     # "default", "nord", ...
    labels=None,      # {layer id: Label}
```

```
    visibility=None,  # {layer id: bool}
)
```

A market diagram on a mosaickit canvas, with axes from 0 to `x_max` and `y_max`. Axis titles are LaTeX math unless they are words; an empty `title` draws none.

`theme` Colours and line widths (Section 15.3).

`palette` Shorthand for `PlotTheme.from_palette(palette)`; `theme` takes precedence when both are given.

`labels` Overrides for built-in labels, by layer id (Section 15.2).

`visibility` Shows or hides layers, by layer id (Section 15.2).

`MarketFigure.add_curves`

`add_curves(⟨demand⟩, ⟨supply⟩, q_max, demand_label="$D$", supply_label="$S$")`

`MarketFigure.add_equilibrium`

`add_equilibrium(⟨result⟩, label="$e^*$", color=None)`

Draw demand and supply from $Q = 0$ to `q_max`, each named at its end, and mark an equilibrium with a filled point and its label.

Each topic adds its own methods, described in its chapter (see Table 3).

Table 3 Topic methods of `MarketFigure`

Method	Section
`add_comparative_statics`	Section 4.3
`add_discrete_curves`, `add_discrete_equilibrium`	Section 5
`add_welfare`, `add_welfare_transition`	Section 8
`add_tax_transform`, `add_tax_comparison`, `add_subsidy_comparison`	Section 9
`add_price_control`	Section 10
`add_trade`	Section 11
`add_externality`, `add_common_resource`	Section 12
`add_minimum_wage`, `add_loanable_funds`	Section 13

`MarketFigure.finalize`

`finalize(legend=False)`

`MarketFigure.save`

`save(⟨path⟩, dpi=150)`

`finalize()` hides guide lines that would cut a shaded welfare region in two (the value stays marked on its axis) and, only with `legend=True`, adds a legend. `save()` writes PNG, SVG or PDF according to the extension and creates missing directories. `close()` is a no-op kept for compatibility.

`MarketFigure.add_layer`

`add_layer(⟨layer⟩)`

`MarketFigure.add_layers`

`add_layers(⟨layers⟩)`

Add any mosaickit layer, for annotations the topic methods do not provide. An axis mark replaces an earlier mark at the same value. The finished scene is available as `fig.scene`.

`MarketFigure.add_metrics`

`add_metrics(⟨values⟩, *, title=None, location="upper right")`

A text box of name–value pairs, for notebooks and debugging. Teaching figures leave it out: the numbers belong in the text.

15.2 Labels and layers

Every line, point, area and piece of text in a figure is a layer with a stable id, such as `market.demand`, `market.demand.label` or `market.welfare.dwl`. The ids follow the structure of the figure: `axes.*` for the axes, `market.<part>` for each element, and a `.label` suffix for the text that names it. `fig.layer_ids` lists every layer; `fig.label_ids` lists the configurable text (labels, axis marks and braces).

`Label` `Label(text=None, visible=None, offset=None, top=None, bottom=None, left=None, right=None)`

New: v0.10.1 *Updated: v0.12.1*

An override for one built-in label. Fields left as `None` keep the package's default. An `offset` in points opts the label out of automatic placement and moves it by exactly that much. `top`, `bottom`, `left` and `right` instead move the label after it has been placed, so principle-viz still picks the spot and you only fine-tune it.

`MarketFigure.configure_label` `configure_label(⟨layer id⟩, label=None, *, text=None, visible=None, offset=None, top=None, bottom=None, left=None, right=None)`

`MarketFigure.configure_layer` `configure_layer(⟨layer id⟩, *, visible)`

New: v0.10.1

Rename, hide or move one label (the id may omit its final `.label`), or show or hide any layer. `hide(*ids)` and `show(*ids)` toggle several at once. Each returns the figure.

The same overrides can be passed when the figure is created, as `labels=` and `visibility=` mappings; they then apply to layers added later as well. Aggregation figures and the canvas functions (`ppf_canvas()`, `public_good_canvas()`, ...) accept the same two arguments. See Figure 27:

```
from principle_viz import Label, MarketFigure

fig = MarketFigure(
    x_max=12, y_max=12,
    labels={
        "market.demand.label": Label(text="Demand"),
        "market.supply.label": Label(text="Supply"),
    },
)
fig.add_curves(demand, supply, q_max=10)
fig.add_equilibrium(eq)
fig.configure_label("market.equilibrium", text="$e^*$")
fig.hide("axes.origin.label")
fig.finalize()
```

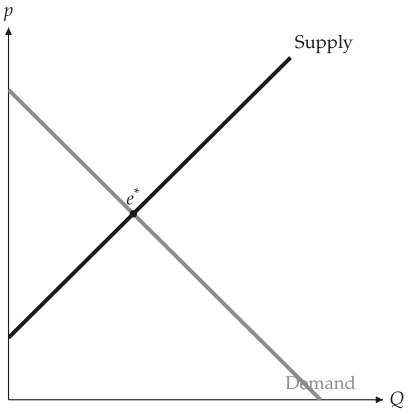


Figure 27 Renamed curves and equilibrium

Nudging a label

A length is a number of points or a string with a unit: `"10pt"`, `"0.5em"` (the label's font size) or `"2mm"`. `top` and `bottom` combine, as do `right` and `left`. A nudge applies after automatic placement, or after an `offset`; later labels still avoid the moved one, and figures without nudges are unchanged. Axis tick labels do not take nudges. The equilibrium label's id is `market.equilibrium`:

```
from principle_viz import Label, MarketFigure

fig = MarketFigure(labels={"market.equilibrium": Label(top="4pt")})
fig.configure_label("market.supply", Label(right="8pt"))
fig.configure_label("market.demand", text="$D_0$", left="0.5em")
```

15.3 Palettes and themes

A `ColorModel` assigns a colour to each economic role; a `PlotTheme` adds line widths and axis options and compiles both into a mosaickit theme. The built-in colour models are the themes every EconViz package shares (Table 4): the same name gives the same look in utility-viz and principle-viz.

`ColorModel` `ColorModel(name, axis_color, label_color, demand_color, supply_color, ...)`

Named colour roles: `axis_color`, `label_color`, `demand_color`, `supply_color`, `baseline_color`, `shifted_color`, `tax_color`, `control_color`, `cs_color`, `ps_color`, `tax_revenue_color`, `dwl_color` and `arrow_color`. Each is `"#hex"` or a mosaickit palette name such as `"blue"`.

Table 4
Built-in colour models

Name	Character and use
`manual`	The greys of the EconViz manuals: black axes, supply and arrows, a grey demand curve, grey surplus regions. For print and handouts
`default`	Demand blue, supply green, deadweight loss red, taxes and controls orange; the surpluses follow the curves. The everyday choice
`colorblind`	The colours of `default` under an explicit name
`monochrome`	Black curves and a grey ladder for the regions, for black-and-white printing
`nord`	Cool colours of the Nord palette, for screens and slides
`paper`	Muted colours, for papers
`presentation`	Brighter colours, with lines 1.5 and the equilibrium dot 1.3 times as large, for projection

A shared theme gives a colour to each role, and `principle_viz.visuals.roles` says which role each element takes: the axis, labels, baseline and arrows take ink, demand primary, supply secondary, shifted curves tertiary, taxes and controls intervention, and consumer surplus, producer surplus, deadweight loss and tax revenue the light, mid, dark and faint fills. The theme's line and marker scales multiply principle-viz's own line widths and equilibrium dot. There is no dark theme.

`list_color_models()` returns the names and `get_color_model(name)` the model; `DEFAULT_COLOR_MODEL`, `COLORBLIND_COLOR_MODEL`, `NORD_COLOR_MODEL` and `MONOCHROME_COLOR_MODEL` are also exported.

`PlotTheme` `PlotTheme(color_model=..., demand_linewidth=3.5, supply_linewidth=3.5, ...)`
`PlotTheme.from_palette(⟨name⟩, **options)`

A colour model with line widths (`demand_linewidth`, `supply_linewidth`, `shifted_linewidth`, `tax_linewidth`, `arrow_linewidth`, `dashed_linewidth`), `equilibrium_marker_size`, and the switches `show_grid`, `show_ticks`, `show_axis_arrows` and `show_origin_label`. `from_palette()` starts from a built-in colour model and sets any of these fields.

Fonts take three fields. `font` is `"sans"` (the default), `"serif"` or `"mono"`; an installed family name such as `"Arial"` is an error. `fonts` is a `mosaickit.TextFonts` that replaces any of the three with font files, the rest keeping mosaickit's DejaVu defaults. `math_font` is a `MathFont` or the path of an OpenType math font, and sets `$...$` through glyphkit. `font_roles` maps a style role to a generic font, for example `{"axes": "mono"}`. Nothing searches the system for fonts, so pass the paths.

```
from mosaickit import TextFonts
from principle_viz import MarketFigure, PlotTheme

theme = PlotTheme(
    font="serif",
    fonts=TextFonts(serif=["fonts/Pagella-Regular.otf", "fonts/Pagella-
Bold.otf"]),
    math_font="fonts/Pagella-Math.otf",
)
fig = MarketFigure(x_max=12, y_max=12, theme=theme)
```

The `palette` argument selects a colour model (see Figure 28):

```
fig = MarketFigure(x_max=12, y_max=12, palette="monochrome")
```

15.4 Settings file

principle-viz reads `econ-viz.toml`, the settings file every EconViz package shares; mosaickit reads and checks it. The top level sets what every package shares: the starting theme (`base`, one of Table 4), the fonts and the axis and label colours. principle-viz's own settings live in the `[principle]` table; the tables of other packages, such as `[utility]`, are skipped.

```
base = "manual"       # a shared theme: manual, default, colorblind,
                      # monochrome, nord, paper or presentation

[font]
text = "serif"
math = "fonts/texgyrepagella-math.otf"

[fonts]
serif = ["fonts/NotoSerifTC-Regular.otf", "fonts/NotoSerifTC-Bold.otf"]
```

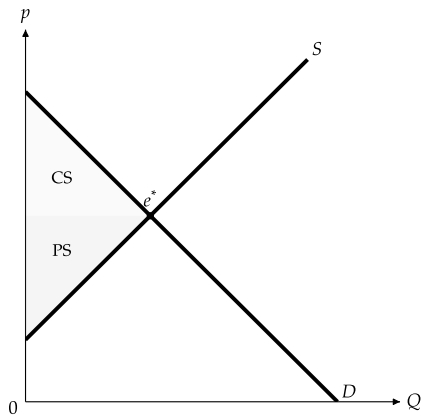


Figure 28
The `monochrome` palette

```toml
[principle]
language = "zh-TW"   # built-in figure text

[principle.color]    # a colour-model role, without "_color"
demand = "#8c8c8c"

[principle.stroke]   # line widths in points
demand = 3.5

[principle.marker]
equilibrium = 6.5    # equilibrium dot diameter in points
```

`base = "manual"` gives the black-and-white look of this manual, so its colours and widths need not be spelled out. `[principle] palette` still sets the colours for principle-viz alone and wins over `base`; `[principle.color]` sets one element. Relative paths are read from the folder of the file.

`Config.load` `Config.use` `default_theme`

```
from principle_viz import Config

Config.load("econ-viz.toml").use()
```

New: v0.12.0 `Config.load()` reads the file and `use()` makes it the default: figures made without `theme=` or `palette=` then use it, and `default_theme()` returns the theme they get. A `theme=` or `palette=` passed to a figure still wins. `mosaickit.econ_toml.use("econ-viz.toml")` loads the file for every installed EconViz package at once.

A mistake raises `mosaickit.ConfigurationError` naming the file and the table, with the closest key when one is close:

```
econ-viz.toml: [principle.color] unknown key 'demnad'; did you mean
'demand'?
```

15.5 Canvases

`Canvas` The mosaickit `Canvas` with the same `labels=`, `visibility=`, `configure_label()`, `configure_layer()`, `hide()`, `show()`, `layer_ids` and `label_ids` as `MarketFigure`. The canvas functions of the PPF, public-good and revenue chapters return it; save it with `save()`, or combine several with `mosaickit.CanvasGrid`.

16 Command-line interface

The `principle-viz` command runs the calculations of this manual from the shell and prints the result as JSON; `--output` also writes it to a file. It draws no figures. Install it as a tool (Section 2.4), or prefix each command with `uv run` inside a project.

```
principle-viz equilibrium \
  --demand-intercept 10 --demand-slope -1 \
  --supply-intercept 2 --supply-slope 1
{
  "q_star": 4.0,
  "p_star": 6.0,
  "is_valid_market": true,
  "notes": []
}
```

16.1 Market arguments

Most commands describe the market by two lines in inverse form, $p = a + bQ$:

`--demand-intercept` Price intercept a of demand.

`--demand-slope` Slope b of demand.

`--supply-intercept` Price intercept a of supply.
`--supply-slope` Slope b of supply.
`--output` Also write the JSON to this file, creating its directory.

16.2 Commands

Table 5 lists the calculation and options of each command; the calculation of each is described in its chapter.

Table 5 CLI commands

Command	Calculation and extra options	Section
`equilibrium`	Linear equilibrium	Section 4
`shift`	Comparative statics: `--demand-delta-intercept`, `--demand-delta-slope`, `--supply-delta-intercept`, `--supply-delta-slope`	Section 4.3
`discrete`	Discrete market: `--demand-values`, `--supply-values`, `--price-rule {midpoint,lower,upper}`	Section 5
`elasticity`	Point elasticity: `--intercept`, `--slope`, `--quantity` arc elasticity with `--q1`, `--p1`	Section 7
`revenue`	Elasticity and revenue schedule: `--samples`	Section 7
`welfare`	Surplus: `--policy {baseline,tax,subsidy,control}` with the options of that policy	Section 8
`report-dwl`	One-row DWL report: `--policy {tax,subsidy,control}`, `--csv`	Section 8
`tax`	`--tax-type {fixed,per_unit,ad_valorem}`, `--amount`, `--tax-on {consumer,producer}`	Section 9
`subsidy`	`--amount`, `--subsidy-to {consumer,producer}`	Section 9
`controls`	`--control-type {ceiling,floor}`, `--control-price`	Section 10
`trade`	`--world-price`, `--tariff`, `--import-quota`, `--quota-rent-recipient`	Section 11
`externality`	`--external-cost`, `--external-benefit`	Section 12
`common-resource`	`--congestion-cost`	Section 12
`public-good`	`--benefit-intercepts`, `--benefit-slopes`, `--cost-intercept`, `--cost-slope`, `--samples`	Section 12
`minimum-wage`	`--labor-demand-*`, `--labor-supply-*`, `--minimum-wage`	Section 13
`loanable-funds`	`--savings-*`, `--investment-*`, `--savings-shift`, `--investment-shift`, `--government-borrowing`	Section 13
`ppf`	`--x-intercept`, `--y-intercept`, `--curvature`, `--x-good`, `--y-good`, `--x-growth`, `--y-growth`, `--samples`	Section 14

`principle-viz <command> --help` lists every option of a command.

```
principle-viz tax \
  --demand-intercept 10 --demand-slope -1 \
  --supply-intercept 2 --supply-slope 1 \
  --tax-type per_unit --amount 1 --tax-on producer
```

The output holds the untaxed equilibrium (`baseline_equilibrium`), the taxed one (`post_tax`) and the changes, as `compare_tax_scenario()` returns them. Here the quantity is 3.5, the consumer price 6.5 and the producer price 5.5.

Change history

Release history of principle-viz, including its releases as `principle-econ`. Documentation-only changes are not listed; each entry is tagged where the feature it describes is documented, following l3doc convention, so the page number is the real page.

v0.12.1 (2026-10-23)

`Label`: `top`, `bottom`, `left` and `right` nudge one label after it has been placed ····· 32

`ColorModel`: The colour models are the shared EconViz themes, so one name gives the same look in every EconViz package; new `paper` and `presentation` ····· 33

`Config`: mosaickit reads and checks the file from the schema principle-viz declares; the shared `base` picks the starting theme ····· 34

v0.12.0 (2026-10-23)

`Config`: `Config.load("econ-viz.toml").use()` reads the settings file every EconViz package shares ····· 34

v0.11.2 (2026-10-22)

`ColorModel`: New `manual` palette: the greys of this manual ····· 33

v0.11.0 (2026-10-21)

`PlotTheme`: New `font`, `fonts`, `math_font` and `font_roles` ····· 34

v0.10.1 (2026-10-03)

General: CI tests Python 3.10–3.13 ····· 2

`demand_aggregation_figure`: The market curve names the demand that is active at a kink ····· 12

`Label`: Every built-in label can be renamed, hidden or moved: `Label`, `labels=`, `configure_label()` and `label_ids` ····· 32

`MarketFigure.configure_layer`: Every layer can be hidden or shown: `visibility=`, `configure_layer()`, `hide()`, `show()` and `layer_ids` ····· 32

v0.10.0 (2026-10-03)

General: Renamed from `principle-econ` to principle-viz, joining the EconViz family; the Python package is `principle_viz` ····· 2

Figures are built on mosaickit (`mosaickit>=0.5.1,<0.6.0`) ····· 2

Project tooling moved from Poetry to uv ····· 2

`principle-viz`: The only runtime dependency is mosaickit ····· 3

`add_comparative_statics`: Redraws only the curve that moved; shifted curves are named D_1 / S_1 (`demand_label` / `supply_label`) ····· 7

Movement arrows are thin, black and dashed ·· 7

`MarketFigure.add_discrete_curves`: Accepts a demand schedule, a supply schedule, or both ·· 9

Open endpoints are drawn with an opaque white face, joined to the next step by a thin dashed riser ····· 9

`market_demand`: Market demand and supply as the horizontal sum of individual linear curves, with `PiecewiseLinear`, `solve_piecewise_equilibrium()` and `piecewise_surplus()` ····· 11

`DiscreteDemand.combine`: `combine()` and `quantity_at()` for discrete schedules ····· 11

`demand_aggregation_figure`: Individual | individual | market figures for demand and supply, linear and discrete, returning an `AggregationFigure` ····· 12

`link_price=True` runs the price line across every panel ····· 12

`MarketFigure.add_welfare`: Regions are named in words (“Consumer surplus”, “CS”, ...) inside, by short name or by callout; `labels=False` shades only ····· 14

Consumer and producer surplus reuse the demand and supply hues; tax revenue is labelled rather than shaded ····· 14

`MarketFigure.add_tax_transform`: Taxed curves are named $S + t$ / $D - t$; the rotation label sits at the arrow’s tail ····· 17

`MarketFigure.add_tax_comparison`: Marks p_d, p_0 and p_s on the price axis with a “Tax” brace; `brace_side` and `notes` ····· 18

`MarketFigure.add_subsidy_comparison`: Subsidy wedge with a “Subsidy” brace, and the subsidy’s cost named ····· 19

`MarketFigure.add_price_control`: Binding controls name the control line, mark p_c, Q_d and Q_s, and brace “Shortage” or “Surplus” on the control line; `gap_brace="axis"` braces it under the quantity axis ····· 20

`MarketFigure.add_trade`: Marks Q_s / Q_d and braces “Imports” / “Exports” under the quantity axis; the policy price is marked $p_w + t$ or p_q ···· 22

`MarketFigure.add_externality`: Social curves are named *MSC* / *MSB*; Q_m and Q^* are marked on the quantity axis; corrective taxes and subsidies span the gap at Q^* ····· 24

`MarketFigure.add_minimum_wage`: Drawn like a price floor: the “Minimum wage” line, $w_{\min}$, L_d / L_s and an “Unemployment” brace ····· 26

`MarketFigure`: Figures are square (6 × 6 in at 150 dpi) on an opaque white background; straight

curves stop at 92% of the price axis ····· 30
Curves are named beside their visible end instead of in a legend ····· 30
Axis titles default to p and Q, set in italic math past the arrow tips; titles use normal weight ·· 30
Equilibrium and policy labels are placed by mosaickit beside their point; crowded quantities are marked on the quantity axis ····· 30
`MarketFigure.finalize`: No legend by default; `finalize(legend=True)` adds one ····· 30
`ColorModel`: The `default` palette takes its hues from mosaickit: demand blue, supply red, deadweight loss teal ····· 33
`PlotTheme`: Supply and demand curves are 3.5 pt wide, axes 1.0 pt, equilibrium points 6.5 pt across ····· 33
v0.1.0 (2026-04-25)
General: Initial release as `principle-econ`: linear markets, taxes, welfare, plotting and a command-line interface ····· 2
`TaxScenario`: Fixed, per-unit and ad valorem taxes, with legal incidence on buyers or sellers ··· 17
`ColorModel`: `default`, `colorblind`, `nord` and `monochrome` colour models ····· 33
`principle-viz`: Command-line interface with JSON output ····· 35

Command Index

A
analyze_common_resource 25
analyze_externality 24
analyze_loanable_funds 28
analyze_minimum_wage 26
analyze_ppf 29
analyze_ppf_growth 29
analyze_public_good 26
analyze_trade 22
B
build_dwl_report 16
build_tax_visual_guide 18
C
Canvas 35
ColorModel 33
comparative_statics 7
compare_linear_ppfs 30
compare_subsidy_scenario 19
compare_surplus 15
compare_tax_scenario 18
compute_arc_elasticity 13
compute_discrete_surplus 10
compute_point_elasticity 13
compute_surplus 15
Config.load 35
Config.use 35
D
default_theme 35
demand_aggregation_figure 12
discrete_demand_aggregation_figure 12
discrete_supply_aggregation_figure 12
DiscreteDemand 9
DiscreteSupply 9
E
elasticity_revenue_canvases 14
elasticity_revenue_schedule 14
evaluate_price_control 21
ExternalityScenario 24
I
IndividualBenefit 26
L
Label 32
Line 6
line_from_inverse 5
LoanableFundsScenario 28
M
market_demand 11
market_supply 11
MarketFigure 5, 30
MarketFigure.add_* 5
MarketFigure.add_common_resource 25
MarketFigure.add_comparative_statics 8
MarketFigure.add_curves 31
MarketFigure.add_discrete_curves 10
MarketFigure.add_discrete_equilibrium 10
MarketFigure.add_equilibrium 31
MarketFigure.add_externality 25
MarketFigure.add_layer 31
MarketFigure.add_layers 31
MarketFigure.add_loanable_funds 28
MarketFigure.add_metrics 31
MarketFigure.add_minimum_wage 27
MarketFigure.add_price_control 21
MarketFigure.add_subsidy_comparison 20
MarketFigure.add_tax_comparison 18
MarketFigure.add_tax_transform 18
MarketFigure.add_trade 23
MarketFigure.add_welfare 16
MarketFigure.add_welfare_transition 16
MarketFigure.configure_label 32
MarketFigure.configure_layer 32
MarketFigure.finalize 5, 31
MarketFigure.save 5, 31
MarketOutcome 14
P
piecewise_surplus 12
PiecewiseLinear 11
PlotTheme 34
PointStatus 29
ppf_canvas 29
ppf_growth_canvas 29
PPFGrowthScenario 29
PriceControlScenario 20
PriceControlType 20
PrincipleVizError 6
ProductionPossibilitiesFrontier 28
public_good_canvas 26
S
ShiftScenario 7
ShiftSpec 7
solve_discrete_equilibrium 9
solve_equilibrium 5, 7
solve_piecewise_equilibrium 12
solve_subsidy_equilibrium 19
solve_tax_equilibrium 17
SubsidyScenario 19
SubsidyTo 19
supply_aggregation_figure 12
T
TaxOn 17

`TaxScenario` 17
`TaxType` 17
`TradeScenario` 22

References

Hardin, G. (1968). The Tragedy of the Commons. *Science*, *162*(3859), 1243–1248. https://doi.org/10.1126/science.162.3859.1243

Mankiw, N. G. (2021). *Principles of Economics* (9th ed.). Cengage Learning.

Marshall, A. (1890). *Principles of Economics*. Macmillan.

Pigou, A. C. (1920). *The Economics of Welfare*. Macmillan.

Samuelson, P. A. (1954). The Pure Theory of Public Expenditure. *The Review of Economics and Statistics*, *36*(4), 387–389. https://doi.org/10.2307/1925895